

Vector Databases

Complete Guide: Storage, Indexing & Querying Embeddings

FAISS (Meta), Pinecone, Qdrant, Weaviate, Chroma | 2024 Reference

Introduction to Vector Databases

Vector databases store, index, and query high-dimensional embedding vectors efficiently. They are a foundational component of modern AI applications including RAG pipelines, semantic search, recommendation systems, and multimodal retrieval.

1. Why Vector Databases?

Traditional databases search by exact matches or range queries on scalar values. Embeddings produced by neural networks (e.g., text-embedding-ada-002, CLIP, E5) encode semantic meaning as points in a high-dimensional space (typically 768–3072 dimensions). Finding semantically similar items requires Approximate Nearest Neighbor (ANN) search, which vector databases are designed to perform at scale.

2. Core Indexing Algorithms

HNSW (Hierarchical Navigable Small World): Graph-based ANN. Builds a multi-layer proximity graph. $O(\log n)$ search complexity. Default in Weaviate, Qdrant, pgvector. Best recall/latency tradeoff.

IVF (Inverted File Index): Quantization-based. Partitions space into Voronoi cells; searches only nearby cells. Used in FAISS. Scalable to billions of vectors with product quantization (IVF+PQ).

LSH (Locality-Sensitive Hashing): Hash-based approach. Maps similar vectors to the same buckets with high probability. Low memory but lower recall than HNSW.

Flat (Brute Force): Exact search over all vectors. Perfect recall but $O(n)$ per query. Suitable for datasets < 1M vectors or when recall is critical.

ANNOY (Approximate Nearest Neighbors Oh Yeah): Tree-based (random projection forests). Used in Spotify recommendation. Read-optimized; index cannot be updated after build.

3. Major Vector Database Comparison

Database	Index	Scalability	Filtering	Best For
----------	-------	-------------	-----------	----------

Pinecone	HNSW/IVF	Managed/PB	Metadata	Managed cloud RAG
Weaviate	HNSW	Distributed	GraphQL	Knowledge graphs
Qdrant	HNSW	Sharded	Payload	High-perf production
Chroma	HNSW	Single-node	Metadata	Local dev / prototypes
Milvus	HNSW/IVF	Distributed	Attributes	Large-scale enterprise
pgvector	HNSW/IVF	PostgreSQL	SQL	Existing Postgres stack
FAISS (Meta)	IVF+PQ/Flat	Library	External	Research / offline
Redis VSS	HNSW/Flat	Redis cluster	Redisearch	Low-latency cache

4. Similarity Metrics

Cosine Similarity: $\cos(u,v) = (u \cdot v) / (\|u\| \|v\|)$ — Direction-based; normalized embeddings. Most common for text.

Dot Product: $u \cdot v = \sum(u_i * v_i)$ — Same as cosine when vectors are unit-normalized. Faster computation.

Euclidean Distance (L2): $\|u-v\| = \sqrt{\sum((u_i - v_i)^2)}$ — Magnitude-sensitive. Used in image embeddings.

Manhattan Distance (L1): $\sum(|u_i - v_i|)$ — Robust to outliers; less common for embeddings.

5. FAISS Quick Start (Python)

```
import faiss
import numpy as np

# Build an index
d = 768          # embedding dimension
n = 100000       # number of vectors

# Create flat L2 index (exact)
index_flat = faiss.IndexFlatL2(d)

# Or HNSW for fast approximate search
index_hnsw = faiss.IndexHNSWFlat(d, 32) # 32 = M (graph degree)

# Add vectors
vectors = np.random.rand(n, d).astype('float32')
index_hnsw.add(vectors)

# Search
query = np.random.rand(1, d).astype('float32')
k = 10 # top-k results
distances, indices = index_hnsw.search(query, k)
```

```
print(f"Top-{k} neighbors: {indices[0]}")

# Save / load index
faiss.write_index(index_hnsw, "my_index.faiss")
loaded_index = faiss.read_index("my_index.faiss")
```

6. Production Considerations

- Normalize embeddings before indexing cosine-based queries
- Use filtered ANN (pre-filter vs post-filter) carefully — pre-filter changes index structure
- Benchmark recall@k, QPS, and memory usage for your specific dataset size
- For billion-scale: use IVF+PQ compression; accepts ~10–15% recall loss for 10x memory savings
- Hybrid search (dense + sparse BM25) consistently outperforms dense-only retrieval